Queue

A ***queue*** is a specialized list with a limited number of operations in which items can only be added to one end and removed from the other. A queue is also known as a ***First-in, first-out (FIFO)*** list. New items are inserted into a queue at the ***back*** while existing items are removed from the ***front***.

A ***queue*** is considered a ***linear*** data structure because its elements are organized in a ***single, sequential order***, typically visualized as a ***horizontal line***. This linear arrangement means elements are ***inserted*** at one end the ***rear*** **or back** and ***removed*** from the other end the ***front*** following the ***FIFO (First In, First Out)*** principle. You cannot directly access elements in the middle; you must process them in the order they were added.

1- Operations on Queue

A ***Queue***, being a linear data structure following the ***First-In, First-Out (FIFO)*** principle, supports a ***limited*** set of operations. Common types of Stack operation includes:

A- Enqueue (Insertion)

Adds an element to the ***rear (end)*** of the queue.

Example: If the queue is **[10, 20]** and you enqueue **30**, it becomes **[10, 20, 30].**

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

B- Dequeue (Deletion)

Remove an element from the ***front (start)*** of the queue.

Example: If the queue is ***[10, 20, 30]*** and you dequeue, it becomes **[20, 30]**.

C- Peak / Front

Get the element at the front of the queue without removing it.

Example: For [10, 20, 30], peek() returns **10**.

D- Rear

Get the element at the rear (last position) without removing it.

Example: For [10, 20, 30], rear() returns 30.

E- IsEmpty

Check if the queue has no elements, Return → **[] or True**

F- IsFull (for fixed size Queue)

Check if the queue is ***full*** or reached to its ***maximum*** Capacity.

If max size = **3** and queue is [10, 20, 30], then isFull() → **True**.

G-Size

Returns the ***total number of elements*** currently in the Queue.

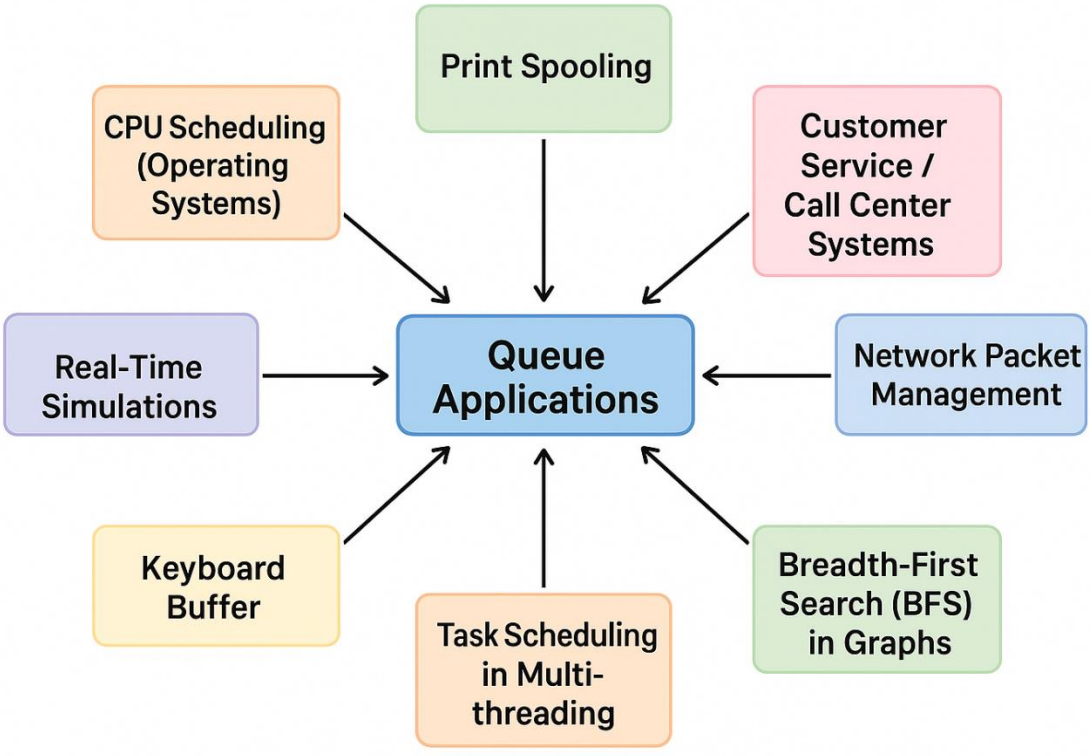
Example: For queue [1, 2, 3], size → 3.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

These operations ensure the ***queue*** maintains its ***FIFO (First In, First Out)*** behavior, allowing access to elements in the order they were added for fairness and efficiency. Some implementations may include ***additional utility*** operations like ***clear*** (to remove all elements), search (to find an element's position), or ***sort*** (to arrange elements in ascending or descending order), but the operations mentioned earlier are the core operations of a queue.

2- Applications of the Queue

Queue have various applications across multiple domains, and a few of them are discussed below:



Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

A- CPU Scheduling (Operating Systems)

Queues manage process scheduling.

Example: Round-Robin scheduling uses a queue to decide which process runs next.

B- Print Spooling

When multiple documents are sent to a printer, they are stored in a print queue.

The printer processes them in the order they arrive (FIFO).

C- Customer Service / Call Center Systems

Customer requests are handled in the order they are received.

Example: Ticket booking, support chat systems.

D- Network Packet Management

Routers and switches use queues to hold packets before transmitting them.

Ensures packets are processed in the correct order.

E- Breadth-First Search (BFS) in Graphs

BFS algorithm uses a queue to explore nodes level by level.

F- Task Scheduling in Multi-threading

Background tasks or jobs are stored in a queue before being executed by worker threads.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

G-Real-Time Simulations

Queues help manage events in traffic systems, airport simulations, etc.

H-Keyboard Buffer

When typing, keystrokes are stored in a queue before being processed.

I- Disk Scheduling

Queues manage read/write requests in disks to optimize performance.

Queues are also the foundation for advanced data structures like ***Priority Queue, Dequeue, and Circular Queue***, which are used in ***OS kernels, networking, databases***, and more.

Queues, using the ***First-In-First-Out (FIFO)*** principle, are essential in ***computer science for managing tasks in sequential order***. They are widely used in ***CPU scheduling***, ***print spooling***, ***I/O buffer handling***, ***task scheduling*** in operating systems***, network packet management, order processing systems, customer service lines, Breadth-First Search (BFS)*** *in graphs****, messaging queues,*** *and* ***real-time event handling***, efficiently handling tasks that require fairness and order.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

3- Implementation of Queue

1) First Step is to Make **Class of Queue**.

```
class Queue:
```

2) Initialization of **Constructor (init function)**

```
def __init__(self, total_values):
    self.queue = []
    self.total_values = total_values
```

3) **Enqueue** Function

```
def enqueue(self):
    if len(self.queue) == self.total_values:
        print("Queue is already full, no space to insert values.")
    else:
        element = int(input("Enter the value you want to insert in the Queue: "))
        self.queue.append(element)
        print("Value inserted successfully!")
        print("Queue is:", self.queue)
```


Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

4) **Dequeue** Function

```
def dequeue(self):
    if not self.queue:
        print("Queue is empty, no value to remove.")
    else:
        e = self.queue.pop(0)  # Remove from front
        print("Removed value is:", e)
        print("Updated Queue is:", self.queue)
```

5) Peak/**Front** Function (For First entered Value)

```
def peek(self):
    if not self.queue:
        print("Queue is empty, no front element.")
    else:
        print("Front value in the Queue is:", self.queue[0])
```

6) Rear/**Back** Function (For last entered Value)

```
def rear(self):
    if not self.queue:
        print("Queue is empty, no rear element.")
    else:
        print("Rear value in the Queue is:", self.queue[-1])
```


Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

7) **IsEmpty** Function

```
def is_empty(self):
    if not self.queue:
        print("Queue is empty.")
    else:
        print("Queue is not empty.")
```

8) **IsFull** Function

```
def is_full(self):
    if len(self.queue) == self.total_values:
        print("Queue is full.")
    else:
        print(f"Queue is not full, and its current length is: {len(self.queue)}")
```

9) **Size** Function (to check length of queue)

```
def size(self):
    print(f"Current size of the Queue is: {len(self.queue)}")
```


Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

10) **Sort** the Queue (Ascending or Descending)

```
def sort(self):
    if not self.queue:
        print("Queue is empty, nothing to sort.")
    else:
        self.queue.sort()
        print("Queue sorted in ascending order:", self.queue)
        self.queue.sort(reverse=True)
        print("Queue sorted in descending order:", self.queue)
```

11) **Display** Function (for print all Values)

```
def display(self):
    if not self.queue:
        print("Queue is empty, no values exist to display.")
    else:
        print("Current Queue (Front → Rear):")
        for i in self.queue:
            print(i)
```


Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

12) Calling Class and Main Menu

```
total_values = int(input("Enter the maximum number of values you want to insert in Queue: "))
queue = Queue(total_values)


while True:
    print("\nPlease Enter the Choice:")
    print("1- Enqueue")
    print("2- Dequeue")
    print("3- Peek (Front)")
    print("4- Rear (Last)")
    print("5- isFull")
    print("6- isEmpty")
    print("7- Size")
    print("8- Sorting")
    print("9- Display Queue")
    print("10- Quit")
```


Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

13) Calling Functions of Queue Class

```
choice = int(input("Please Enter your Choice: "))
    if choice == 1:
        queue.enqueue()
    elif choice == 2:
        queue.dequeue()
    elif choice == 3:
        queue.peek()
    elif choice == 4:
        queue.rear()
    elif choice == 5:
        queue.is_full()
    elif choice == 6:
        queue.is_empty()
    elif choice == 7:
        queue.size()
    elif choice == 8:
        queue.sort()
    elif choice == 9:
        queue.display()
    elif choice == 10:
        print("Exiting program. Goodbye!")
        break
    else:
        print("Please enter the correct option to proceed further.")
```


Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

4- Practice Questions

Below are the ***5 practice questions*** based on the ***Queue*** implementation that will help you to learn and practice ***Queue*** effectively:

1. Ticket Counter Simulation

A ***ticket counter*** serves customers in the ***order*** they arrive. Write a program using a Queue to:

1) Add customers as they arrive **(enqueue).**
2) Serve customers one by one **(dequeue).**
3) Show the next customer to be served **(peek).**
4) Display the remaining customers.

2. Printer Spooler System

A ***printer*** can only ***print one job at a time***. Create a program where:

1) **Print** jobs (documents) are **added** to a queue.
2) When the printer finishes one job, it **removes** it from the queue.
3) The system should show which job is currently **printing** and which jobs are **waiting**.

3. Hospital Patient Queue

A hospital emergency room ***handles patients*** in the order they come. Implement a system to:

1) Add new patients to the queue.
2) Remove and display the patient currently being treated.
3) Show the list of patients still waiting.

4. Call Center Simulation

A call center handles incoming customer calls on a first-come, first-served basis using a queue. Write a program that:

1) **Adds callers** to the queue when they call (accept the caller's name as input).
2) **Removes** a caller from the queue when their issue is resolved.
3) Displays the next caller to be served without removing them from the queue.
4) Displays the complete list of callers currently waiting in the queue.

Hint: Use enqueue for adding, dequeue for removing, and peek for the next caller.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk

5. Queue-based Task Scheduler

A background task scheduler runs tasks in the order they arrive (FIFO). Write a program that:

1) Adds a task to the queue (***accept task name or description as input***).
2) Executes and removes the task at the ***front*** of the queue (simulates running the task).
3) Displays the current number of tasks left in the queue after each operation.
4) Shows all pending tasks in the queue at any time.

Hint: Use queue operations and count remaining elements after each step.

Instructor: Dr. Hafiz Muhammad Tayyab Khushi
Muhammad.tayyab@superior.edu.pk